



Inferring preferences from demonstrations in multi-objective reinforcement learning

Junlin Lu¹ · Patrick Mannion¹ · Karl Mason¹

Received: 24 July 2023 / Accepted: 8 August 2024 / Published online: 28 September 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024

Abstract

Many decision-making problems feature multiple objectives where it is not always possible to know the preferences of a human or agent decision-maker for different objectives. However, demonstrated behaviors from the decision-maker are often available. This research proposes a dynamic weight-based preference inference (DWPI) algorithm that can infer the preferences of agents acting in multi-objective decision-making problems from demonstrations. The proposed algorithm is evaluated on three multi-objective Markov decision processes: Deep Sea Treasure, Traffic, and Item Gathering, and is compared to two existing preference inference algorithms. Empirical results demonstrate significant improvements compared to the baseline algorithms, in terms of both time efficiency and inference accuracy. The DWPI algorithm maintains its performance when inferring preferences for sub-optimal demonstrations. Moreover, the DWPI algorithm does not necessitate any interactions with the user during inference—only demonstrations are required. We provide a correctness proof and complexity analysis of the algorithm and statistically evaluate the performance under different representation of demonstrations.

Keywords Multi-objective reinforcement learning · Preference inference · Dynamic weight multi-objective agent

1 Introduction

Many decision-making tasks are designed with the primary goal of optimizing performance in achieving a singular objective, such as enhancing game scores [1], reducing energy consumption [2], and ensuring appropriate robotic functionality [3], among others. Nonetheless, in practical scenarios, more decision-making tasks feature multiple objectives. This complexity introduces the necessity for a balanced trade-off among these objectives. Given the variability in user preferences regarding objectives, the introduction of a preference factor, typically represented by

a linear weight vector [4], is essential for the comparative evaluation of distinct solutions.

Multi-objective reinforcement learning (MORL) is the paradigm to solve such multi-objective decision-making (MODM) problems, by considering the preference factor. If the preference factor is known beforehand, a standard policy improvement approach can be used [5–7]. This, however, cannot work properly without knowing the preference beforehand, and it needs to be retrained from scratch when the preference changes. Though there are more advanced works that can work with dynamic preferences [8–10], it cannot give the correct solution when the user cannot say out true preference. It is challenging for the user to numerically specify the preference corresponding to their true thoughts. For example, when selecting stocks for a portfolio, the portfolio manager selects stocks based on their weighting of minimizing risk and maximizing potential future profits. He/she might want to give a higher weighting to maximize potential future profits, but should the weight be 0.7, 0.8, or some other number? Setting weights this way is unintuitive and must typically be done by trial and error. Moreover, even if the user can provide an

✉ Junlin Lu
j.lu5@nuigalway.ie

Patrick Mannion
patrick.mannion@universityofgalway.ie

Karl Mason
karl.mason@universityofgalway.ie

¹ School of Computer Science, University of Galway,
University Road, Galway H91 TK33, Ireland

approximate numerical preference, a small error in their preference can result in a significantly different policy being learned and executed which may lead to a sub-optimal solution.

While it may be challenging for users to express preferences, their behavior/demonstration can provide insights into their true thoughts. The process of inferring preference from demonstrations is known as preference inference (PI) [11–14]. By leveraging user demonstration to infer their true thoughts, we can make more effective and user-friendly MODM systems.

Although there is some previous work on the PI for MODM, inferring preferences from demonstrations is still challenging and has yet to receive enough attention in the literature. Many PI methods involve active learning approaches, which require feedback from users [15–18]. Other methods use ideas from inverse reinforcement learning (IRL) [19, 20] to infer preferences from demonstrations and heuristically search for the correct weight vector in the preference space [21, 22], which can be computationally expensive and suffer from low accuracy without optimal demonstrations. MORL approaches that learn the parameterized policy set can also be used to infer user preference [9, 23]. These PI paradigms either suffer from high computing overload, frequent interaction with the user, or brittleness when sub-optimal demonstrations are given. It would therefore be advantageous to develop a new method that can perform autonomous query-free PI at a low computational cost and is robust against sub-optimal demonstrations.

We propose a dynamic weight-based preference inference (DWPI) algorithm that aims to train a PI model that is free of querying user and robust against sub-optimal demonstrations. Our method does not need the user to provide any training data. The training data are all generated by a dynamic weight MORL (DWMORL) agent. We evaluate the proposed algorithm in three different environments featured with various number of objectives, namely Convex Deep Sea Treasure (CDST) [24], Traffic, and Item Gathering [8]. The DWPI algorithm involves training a deep neural network model for PI using a feature set of demonstration and a target set of preference weight vectors generated by the DWMORL agent in simulations. Compared to existing approaches, the DWPI algorithm avoids the computational overhead and requires no user feedback than PI paradigms with active learning. Additionally, the DWPI algorithm is more robust than other methods in the literature as it can infer preferences correctly even if the demonstration is stochastic and sub-optimal. The main contributions of this paper are:

- We propose the DWPI algorithm, a time-efficient, robust, query-free, and high-accuracy method for

inferring preferences from demonstrations (including sub-optimal demonstrations) in MORL settings. After the model is trained, the model only requires a forward pass to infer the preference for a given demonstration.

- The DWPI algorithm's generalization capability is assessed across three distinct environments, each featuring a varying number of objectives, to demonstrate its adaptability and effectiveness.
- We provide a proof of correctness for the utilization of a feedforward neural network (FNN) in the PI process from demonstrations. Additionally, we perform a complexity analysis of our algorithm and introduce a metric to evaluate the accuracy of the inference process.
- An energy-based model is introduced to deliberately generate sub-optimal demonstrations, thereby enriching the training dataset for the PI model, and enhancing its learning potential.
- The performance of the DWPI algorithm is evaluated under various demonstration representations, underscoring its versatility and efficiency in processing diverse data inputs.

The remainder of this paper is organized as follows. We outline the necessary background knowledge for this paper in Sect. 2. In Sect. 3 we present the formal model for the DWPI algorithm. In Sect. 4, we illustrate the design of our experiment. The results of the experiments are demonstrated and discussed in Sect. 5. In Sect. 6, we conclude the paper and propose some future research directions that have arisen from this work.

2 Background and related work

2.1 Multi-objective reinforcement learning

Multi-objective reinforcement learning (MORL) is a branch of reinforcement learning that focuses on decision-making problems with multiple objectives [4, 7, 25, 26]. In MORL, an agent interacts with the environment and receives a reward vector composed of multiple objective values, rather than a single scalar reward like in single-objective reinforcement learning (SORL). However, this increases the computation complexity of training. To make training more manageable, reward scalarization is often used in MORL. This involves mapping the reward vector to a scalar value using a scalarization function, also known as a utility function. The utility function $u : \mathbb{R}^d \rightarrow \mathbb{R}$ is given below,

$$V_u^\pi = u(V^\pi) \quad (1)$$

where the V^π is the multi-objective value vector when adopting policy π and V_u^π is the scalarized value of policy π .

Linear scalarization is a commonly used method in MORL and has been applied in various studies, where weights are assigned to each of the objectives to compute a weighted sum [7, 23, 27–29]. The weights reflect the relative importance of the different objectives. The resulting scalar value (utility) can be used to compare different policies.

The convex coverage set ($\text{CCS}(\Pi)$) is a subset of the Π , where Π is the set of all possible policies. $\text{CCS}(\Pi)$ contains optimal policies for every linear weight vector [4]. Within the scope of using linear utility function, $\text{CCS}(\Pi)$ contains all necessary information for optimal policy set, i.e., the $\text{CCS}(\Pi)$ represents the set of Pareto optimal solutions that are guaranteed optimal for any linear combination of the objectives.

2.2 Preference inference

Previous studies have mainly focused on uncovering the user's unknown preferences within MODM. These methodologies are broadly categorized into approaches that leverage queries to elicit preferences [15–18], inverse reinforcement learning paradigm (IRL) [21, 22], and using a learned policy set to do the inference [9, 23]. We start with the query-based methods.

Incremental Weight Elicitation (IWE): Benabbou et al. proposed a method utilizing a vector-valued state-space graph [15]. This approach begins by identifying a set of potential preferences, followed by employing incremental queries to narrow down the volume of candidate preferences until an approximate optimal solution is identified.

Active Learning Method—Ordered Preference Elicitation (OPE): Zintgraf et al. introduced an active learning strategy termed as ordered preference elicitation [16]. This method presupposes the obscurity of the user's intrinsic preference and adopts a Gaussian process to model this latent preference. It features three querying strategies: pair comparison, ranking, and clustering, aimed at aligning the policy with the user's underlying preferences.

Regret-Based Incremental Preference Elicitation (RIGA): Employing a genetic algorithm, this approach, as discussed by Benabbou et al. [17], also utilizes incremental querying. However, unlike the IWE method, RIGA applies incremental queries to select instances from a set rather than to diminish the set of candidate preferences.

Comparative Feedback/Query (CF) for Eliciting Preference: Recent work by Shao et al. [18] advances the elicitation process by soliciting user feedback on pairs of policies or through weighted representative demonstration

feedback. This method facilitates preference elicitation by enabling direct comparison between different policy outcomes.

It is not explicitly stated in the original contributions whether these query-based methods work for sub-optimal demonstrations. However, they might be able to find out the underlying preferences even from sub-optimal demonstrations as the query involves the user iteratively updating the solution.

Another branch is to use the IRL paradigm. Ikenaga et al. [21] proposed a PI approach, i.e., projection method (PM) based on the inverse reinforcement learning (IRL) paradigm. Their model starts from a heuristic preference and compares the expected feature (e.g., cumulative rewards) of an expert policy, and the expected feature obtained using the inferred preference. The distance between the ground truth preference and the inferred preference is measured. The inference process is considered correct once the reward expectation of the inferred preference converges to the expectation of the ground truth preference. However, the inferred preference is from random sampling from the preference space, and at each round of inference, the RL agent must be trained from scratch each time to obtain the reward expectation of the inferred preference. This can be time-consuming for high-dimensional preference spaces. To address this, Takayama et al. [22] proposed a more efficient method based on deep reinforcement learning (DRL) and multiplicative weights apprenticeship learning (MWAL). Rather than randomly sampling, this method updates the inference along the direction indicated by the difference between the feature expectation of the expert policy and the feature expectation of the inferred preference. However, training an RL agent from scratch is still one of the bottlenecks for this approach. Their approaches have a further limitation in that they do not take into account sub-optimal demonstrations, where the demonstrator may not possess complete expertise for the problem at hand.

Barrett et al. [23] have developed convex hull value iteration (CHVI). This method primarily focuses on learning to solve MORL tasks, with the PI task considered an auxiliary application. The CHVI method facilitates PI through a systematic comparison between the agent's action selections and the acquired knowledge of Q-values, enabling the identification of the preference weight set that aligns with the agent's decisions. By applying intersection operations on these weight constraints, the method arrives at a refined inference of the preference vector.

The envelope MOQ-learning algorithm (EMOQL) proposed by Yang et al. introduces a mechanism whereby the agent learns an optimal policy that accommodates all conceivable preferences. Their research posits that EMOQL is capable of elucidating the underlying

preference vector. The assumption is that there exists a latent preference factor capable of converting the reward vector to a scalar, r . By maintaining a constant policy set and traversing the preference space, it is possible to determine which preference maximizes the cumulative reward scalar with signals when scalarized by this hidden preference [9]. The identified preference is thus considered the most accurate representation of the latent preference factor.

In conducting a comparative analysis of PI methods, as summarized in Table 1, two key criteria are employed for evaluation: the dependency on querying to elicit preference, and the capability to handle sub-optimal demonstrations.

Specifically, it distinguishes between methods that require active engagement with users to refine PIs through queries and those that autonomously infer preferences from observed behaviors. Additionally, it assesses the resilience of these methods in deriving accurate preference insights from demonstrations that may not represent the optimal policy execution. This highlights the adaptability to real-world complexities where ideal demonstration data may be unavailable. This comparative framework facilitates an understanding of the strengths and limitations inherent to each PI approach.

2.3 Problem statement

As highlighted, the deployment of MORL agents in real-world decision-making necessitates an understanding of the user's genuine preferences regarding the weighting attributed to various objectives. Regrettably, the articulation of numerical preferences, which typically manifest as a linear weight vector [30–33], is not inherently intuitive for the user. In adverse scenarios, even when a user is capable of specifying a distinct preference, this preference must be sufficiently precise, as minor discrepancies can lead to the adoption of an inappropriate policy. Such obstacles hinder

the application of MORL methods in practice. This motivates the study in PI methods for MORL scenario.

Though there are PI methods in the literature, as we mentioned in Sect. 2.2, these approaches often require querying the user, face challenges with computational complexity in scenarios involving high-dimensional preference spaces, or struggle to accommodate sub-optimal demonstrations.

Our target is to introduce a PI model designed to surmount the previously mentioned limitations. We employ a dynamic weight multi-objective reinforcement learning (DWMORL) agent [8] to produce data for the inference model. The advantage of self-generating training data is the elimination of the need for user queries during the training phase. Once the DWMORL agent has been adequately trained, the inference procedure becomes a straightforward feedforward operation, irrespective of the number of objectives involved. To facilitate the inference of preferences from sub-optimal demonstrations, we enable the DWMORL agent to generate sub-optimal data. This approach allows for the augmentation of the training set with sub-optimal demonstrations. We assess our methodology across three MORL benchmarks: CDST [24], Traffic, and Item Gathering [8].

We wish to clarify that the term “preference.” The term used in our discourse is specifically defined within the context of MORL, distinguishing it from its usage in discussions surrounding the integration of IRL and SORL. In the work of Hejna et al. [34], they mentioned “preference” as well; however, it is used to rank the demonstration of the robot, to navigate it toward a better policy. It is different from the “preference” in our work where it is a linear weight vector to balance between several known objectives. The target of this method is also different from ours. They focus on involving human feedback to improve the task performance while we aim at inferring the users' feature of preference which can be used for opponent modeling, etc.

Table 1 Comparative analysis

Method	Free of querying users	Robust to sub-optimal demos	References
IWE	×	✓	[15]
OPE	×	✓	[16]
RIGA	×	✓	[17]
CF	×	✓	[18]
PM	✓	×	[21]
MWAL	✓	×	[22]
CHVI	✓	×	[23]
EMOQL	✓	×	[9]
DWPI (ours)	✓	✓	–

3 DWPI algorithm

In this section, we commence by presenting the DWMORL agent, which can adjust to changing priorities across multiple objectives. It plays a pivotal role in producing the training dataset for the DWPI model. Subsequently, we introduce the concept of truncated Q soft Q action sampling, which is employed to create sub-optimal demon-

indexed by a preference weight vector, to maintain a record of action-state values.

At the beginning of each episode, a preference weight $\mathbf{w}$ is randomly sampled from $\mathcal{W}^\eta$. The DWMOTQ agent then interacts with the environment and updates the Q table associated with $\mathbf{w}$ until it converges.

Algorithm 1 DWMOTQ Algorithm

```

Initialize:
Env (environment),  $\alpha$  (learning rate),  $\gamma$  (discount factor)
 $\epsilon$  (exploration factor),  $\pi_\epsilon$  (action sample policy conditioned on  $\epsilon$ )
M (maximum number of training episodes), H (horizon length)
for each  $\mathbf{w}$  in  $\mathcal{W}^\eta$  do
  Reset Env
  Initialize a Q table  $Q_{\mathbf{w}}$ 
  for i  $\leftarrow$  1 to M do
    for t  $\leftarrow$  1 to H do
      Action sample:  $a_t \sim \pi_\epsilon(s_t)$ 
      Get reward signal and next state from environment:  $\mathbf{r}_t, s_{t+1} = E(a_t)$ 
      Scalarize reward:  $r_t = \mathbf{r}_t \cdot \mathbf{w}$ 
      Calculate TD error:  $\delta_t = r_t + \gamma \cdot \max_a Q_{\mathbf{w}}(s_{t+1}, a) - Q_{\mathbf{w}}(s_t, a_t)$ 
      Update Q evaluation:  $Q_{\mathbf{w}}(s_t, a_t) \leftarrow Q_{\mathbf{w}}(s_t, a_t) + \alpha \cdot \delta_t$ 
    end for
  end for
  Save  $Q_{\mathbf{w}}$ 
end for

```

strations to enhance the training dataset further. Following this, we elucidate the DWPI algorithm. The section continues with a theoretical analysis that encompasses both a proof of correctness and an analysis of complexity, and the bidirectional mapping between demonstrations and preferences. In Table 2 we put a summary of the symbols used in this section.

3.1 Dynamic weights MORL agent training

We employed the methodology proposed by Kallström et al. [8] to train the DWMORL agent. Specifically, we adapted this approach to implement both tabular and DRL Q-agents, which are, respectively, referred to as the DWMOTQ (dynamic weights tabular Q-learning) and DWMODQN (dynamic weights deep Q network) algorithms. We partitioned the preference weight space $\mathcal{W}$ into a discrete subset $\mathcal{W}^\eta$ using a specific granularity η (e.g., $\eta = 0.01$ for DST), such that $\mathcal{W}^\eta \subseteq \mathcal{W}$.

3.1.1 Dynamic weights multi-objective tabular Q-learning

The DWMOTQ algorithm is outlined in Algorithm 1. The DWMOTQ agent keeps a collection of Q tables, each

The DWMOTQ algorithm is an effective approach for solving low-dimensional preference and discrete state-space problems. It demonstrates success in environments like Deep Sea Treasure where the preference space is only two-dimensional and the state space is one-dimensional, i.e., the position. In terms of convergence speed in such simple environments, DWMOTQ outperforms DWMODQN which sometimes even fails to converge due to the high level of discreteness of the environment.

Nonetheless, in scenarios characterized by image-based state representations and an expanded preference space dimensionality, the computational feasibility of the DWMOTQ algorithm is compromised. As a solution, we use the DWMODQN algorithm, which serves as an alternative strategy to address this challenge.

3.1.2 Dynamic weights multi-objective deep Q network

We adopted the DWMODQN algorithm as an alternative method for approximating the Q function conditioned on the preference weight vector. The DWMODQN algorithm is built upon the deep Q network (DQN) [1] by integrating dynamic weights across multiple objectives [8].

Table 2 Notation and Nomenclature of Algorithmic Terms

Symbol	Meaning	Symbol	Meaning
Env	Environment	α	Learning rate
γ	Discount factor	ϵ	Exploration factor
π_{ϵ}	Policy conditioned on ϵ	M	Number of episodes
H	Horizon length	$\mathbf{w}$	Preference weight vector
$\mathcal{W}$	Preference weight simplex	η	The granularity discretize $\mathcal{W}$
$\mathcal{W}^{\eta}$	Discretized subset of	$Q_{\mathbf{w}}$	Q table labeled by $\mathbf{w}$
a_t	The action sampled at time step t	s_t	The state at time step t
$\mathbf{r}_t$	The reward vector at time step t	r_t	The reward scalar at time step t
δ_t	The TD-error at time step t	π_{soft}^+	Truncated soft policy
π^*	The optimal policy	δ	Policy equivalence threshold
θ	The Q network	θ_-	The target Q network
$\pi_{\epsilon, \theta, \mathbf{w}}$	Policy conditioned on $\epsilon, \theta, \mathbf{w}$	$\mathcal{D}$	Replay memory
N_{min}	Volume threshold of $\mathcal{D}$	ϕ	The PI model
π	DWMORL agent(policy)	$\tau_{\mathbf{w}_i}$	Demonstration based on $\mathbf{w}_i$
$\mathcal{L}$	Loss function	∇	Gradient operator
β	Boltzmann distribution temperature	N	Number of actions retained
a	Augmented factor for training data	E	DWPI train epoch number
$\mathbf{R}^n$	n -dimensional reward space	n_l	l th layer neuron number

At the beginning of each episode, the DWMODQN agent samples a preference weight vector $\mathbf{w}$ as part of its state input and is trained on a high-granularity partitioned discrete subset $\mathcal{W}^{\eta}$ of the preference space. The DWMODQN can learn a single policy that can handle dynamic preference weights without retraining. The DWMODQN algorithm is outlined in Algorithm 2.

Algorithm 2 DWMORL Algorithm

3.2 Truncated soft Q action sampling

We retained the training process using a deterministic Q estimator of the DWMORL agent and proposed a stochastic sub-optimal action sampling method for demonstration generation, i.e., truncated soft Q action sampling from the trained Q estimators. This is similar to the idea of using an energy-based policy method [35, 36]. However, unlike energy-based policy methods that utilize entropy to improve exploration efficiency and policy

Initialize: Env (environment), α (learning rate), γ (discount factor)
 θ (Q-network), θ^- (target Q-network), ϵ (exploration factor)
 $\pi_{\epsilon, \theta, \mathbf{w}}$ (action sample policy conditioned on ϵ, θ and preference weight $\mathbf{w}$)
 M (maximum number of training episodes), H (horizon length)
 $\mathcal{D}$ (replay memory), N_{min} (minimum filled capacity of $\mathcal{D}$ to trigger training)
for $i \leftarrow 1$ **to** M **do**
 Preference weight vector sample $\mathbf{w} \sim \mathcal{W}^{\eta}$
 for $t \leftarrow 1$ **to** H **do**
 Action sample: $a_t \sim \pi_{\epsilon, \theta, \mathbf{w}}(s_t)$
 Get reward signal and next state from environment: $\mathbf{r}_t, s_{t+1} = E(a_t)$
 Scalarize reward: $r_t = \mathbf{r}_t \cdot \mathbf{w}$
 Add $(s_t, a_t, s_{t+1}, r_t, \mathbf{w})$ to $\mathcal{D}$
 end for
 if $i > N_{min}$ **then**
 Sample experience batch from $\mathcal{D}$
 Calculate TD error: $\delta_t = r_t + \gamma \cdot \max_a Q_{\theta^-}(s_{t+1}, a, \mathbf{w}) - Q_{\theta}(s_t, a_t, \mathbf{w})$
 Update DQN weights: $Q_{\theta}(s_t, a_t, \mathbf{w}) \leftarrow Q_{\theta}(s_t, a_t, \mathbf{w}) + \alpha \cdot \delta_t$
 end if
end for

robustness, truncated soft Q action sampling aims to generate sub-optimal demonstrations intentionally.

The sub-optimal demonstration assumption has been pointed out in literature [37], i.e., “The assumption where the user can always provide at least stochastically optimal demonstrations, does not hold in most real-life scenarios.” The method proposed in the referenced work seeks to enhance the efficiency of robotic policy learning by amalgamating sub-optimal demonstrations. Their approach diverges significantly from ours, as our primary objective is to deduce a numerical, vectorized preference factor from the user’s demonstrations, aiming to capture and quantify the underlying preferences.

Furthermore, the focus of the Chen et al. work does not encompass MORL, which is central to our investigation. Our work is specifically tailored to environments characterized by multiple objectives, where understanding and integrating user preferences is crucial for developing policies that accurately reflect the diverse and often competing goals inherent to such settings.

Once fully trained, Q estimators, such as a Q table or a DQN, possess the capability to precisely evaluate state-action pairs. Although in most literature the trained Q estimator simply selects the optimal action, we argue that the full capacity of Q estimators is often underestimated if they are only used for this purpose. Once Q estimators are trained, they can provide a fair comparison between actions, which may include sub-optimal solutions that are also worth considering. The DWMORL agent is trained using ϵ -greedy, and this truncated soft Q action sampling is only used during demonstration generation.

Algorithm 3 Truncated Soft Q Action Sampling

Initialize:

Temperature of the Boltzmann distribution of action: β

Sample action:

Estimate Q vector $\mathbf{Q}(s_t, \cdot)$ for state s_t

Set the last $|\mathcal{A}| - N$ values of $\mathbf{Q}(s_t, \cdot)$ as $-\infty$ and get truncated Q vector $\mathbf{Q}^+(s_t, \cdot)$

Calculate the probability of action sampling from $\mathbf{Q}^+(s_t, \cdot)$:

$$\pi_{soft}^+(a_i|s_t) = p(a_i|s_t) = \frac{e^{-\beta \cdot \mathbf{Q}^+(s_t, a_i)}}{\sum_{j=0}^{|\mathcal{A}|-N} e^{-\beta \cdot \mathbf{Q}^+(s_t, a_j)}}$$

Sample an action from the truncated soft policy $a_t \sim \pi_{soft}^+(s_t)$

The truncated Q vector is mapped to the probability space, i.e., a Boltzmann distribution. The temperature parameter determines the gap between the probabilities. N actions are retained for sampling from the action space $\mathcal{A}$. While the truncated soft policy π_{soft}^+ is not as guaranteed to be an optimal policy π^* , it is in most cases a good policy as the truncation eliminates most adverse actions while

approximately maintaining the distribution of the quality of actions. By generating sub-optimal demonstrations, the training dataset is augmented. This increases the robustness of the inferring model.

In our experiment, we utilize a combination of optimal and sub-optimal demonstrations to train the DWPI model. Specifically, we make 50% of the demonstration set optimal demonstrations and the remaining 50% sub-optimal demonstrations. To further augment the training data, we repeat each demonstration 50 times. The demonstration generation process and the training process of the DWPI model are illustrated in Fig. 1.

During the evaluation, we average the cumulative reward vector of 100 demonstrations for each preference weight and pass them to the inferring model. We ensure that the same ratio of optimal and sub-optimal demonstrations is maintained across all preference vectors.

Notably, in literature, there are works that also try to insert noise into policy to get sub-optimal demonstrations, i.e., D-REX [38]. However, our method is fundamentally different from D-REX. We deliberately generate sub-optimal demonstrations through truncated soft Q action sampling, aiming to introduce the “second/third best action” rather than relying on random noise that was adopted in D-REX. This approach is designed to more accurately simulate real-world conditions where demonstrations for PI may not always represent the optimal action. Thus, the application of noise in our research serves a distinct purpose, emphasizing the strategic generation of sub-optimal demonstrations to enrich the dataset for a more accurate and comprehensive PI process.

Another work that mentioned sub-optimal demonstra-

tions is [39]. A key distinction between our work and that of Beliaev et al. lies in the setting and application of sub-optimal demonstrations. They focus on extracting valuable insights from sub-optimal demonstrations to enhance policy learning within a singular objective framework. In contrast, our work is situated in a MORL context, where the complexity of balancing competing objectives inherently alters the utility and interpretation of sub-optimal

demonstrations. Our approach seeks not to filter or distill behaviors but to infer and quantify underlying user preferences across multiple objectives, a dimension not addressed by the single-objective focus of the prior study.

Moreover, the targets of our work and their work diverge significantly. The aim of our work is not the optimization of policy learning from heterogeneous demonstrators. Instead, our work tackles the challenge of understanding and modeling the preference vectors that drive demonstrator behavior in multi-objective settings. This involves a fundamentally different application of sub-optimal demonstrations, one that serves to illuminate the diverse priorities and decision-making criteria of users rather than to improve policy performance.

3.3 DWPI algorithm

Our methodology distinguishes from previous literature by eliminating the necessity for any queries or comparisons among demonstrations, as well as the need to retrain a MORL agent from scratch on each occasion. The DWPI model can also infer demonstrator's preference from sub-optimal demonstration.

A trained DWMORL agent can generate the optimal demonstration based on its preference weights, a process referred to as “from preference factors to demonstrations.” In contrast, the reverse process, “from demonstrations to preference factors,” can be approached as a regression problem. The demonstration features can serve as the input and the preference weights act as the output labels. This regression model can be implemented with a supervised learning paradigm, as detailed in Algorithm 4. Initially, we iterate the preference weight space and pass weights into the DWMORL agent to generate demonstrations (the demonstration generation phase). Subsequently, we proceed to train the inference model as a regression model with the demonstrations (inference model training phase). The loss function used for training is the mean squared error (MSE).

Algorithm 4 Dynamic Weight Preference Inference Training Algorithm

3.4 Theoretical analysis

In this section, a theoretical analysis is conducted. We first present a proof of correctness, demonstrating that the DWPI algorithm can be effectively implemented using an FNN. Subsequently, we give the DWPI algorithm's computational complexity. Following this, we showcase how feasibility of inferring preference weights from optimal demonstrations. Subsequently, we relax the assumption of optimal demonstrations to include sub-optimal ones.

3.4.1 Correctness proof for DWPI

We present the correctness proof of using an FNN to do PI tasks from demonstrations. Consider a FNN composed of an input layer, several hidden layers, and an output layer. The network aims to approximate the mapping function: $f: \mathbf{R}^n \rightarrow \mathcal{W}$, all elements on $\mathcal{W}$ are non-negative and sum to 1 [4] and $\mathbf{R}^n$ is the continuous reward space.

Universal Approximation Theorem [40, 41] states that, given any continuous function on a compact set and any error greater than zero, there exists a sufficiently large FNN that can approximate this function within an error less than some threshold over the compact set.

In the context of the DWPI algorithm, the continuous function in question maps continuous reward vectors to the preference simplex space. The Universal Approximation Theorem ensures that there exists a neural network structure capable of approximating such a mapping function. This establishes the validity of using an FNN to map demonstrations to preferences, providing a theoretical foundation for DWPI.

3.4.2 Complexity analysis

In this part, we present the computational complexity analysis for Algorithm 4. The algorithm is separated into two primary phases: data collection and training. During the data collection phase, the computational complexity is

Initialize:

PI model ϕ , DWMORL agent π

Demonstration Generation:

for $\mathbf{w}_i$ in $\mathcal{W}^\eta$ **do**

 Sample demonstrations by the DWMORL agent: $\tau_{\mathbf{w}_i} \sim \pi$

 Add $\tau_{\mathbf{w}_i}$ to training set, add $\mathbf{w}_i$ to label set

end for

Inference Model Training:

while model ϕ not converge **do**

 Sample n entries from training set, calculate loss $\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}_i - \phi_i(\tau))^2$

 Update ϕ model: $\phi \leftarrow \phi - \alpha \nabla \mathcal{L}$

end while

directly proportional to the granularity η , which denotes the level of discretization applied to the preference simplex. For each discretized element of the preference simplex, the agent is required to execute its policy for an episode. The horizon of the episode and the number of episodes the MORL agent is trained, denoted by H and M emerge as critical factors influencing the overall complexity. Consequently, the computational complexity of the data collection phase is expressed in terms of $\mathcal{O}(\eta HM)$, indicating that the time complexity scales with both the granularity of the preference simplex discretization and the length of the episode.

In the training phase, the computational complexity is influenced by the architecture of the FNN utilized. We assume this FNN is full-connected and has L layers, and the l th layer has n_l neurons. On the l th layer, the computational complexity is $\mathcal{O}(n_l n_{l-1})$; therefore, the whole network feedforward computational complexity is $\mathcal{O}(\sum_{l=1}^L n_l n_{l-1})$, the backward propagation has similar complexity to the feedforward step. Given that there are $n\eta$ demonstrations in the training set, where a is the augmented factor of the training dataset, and the training involves E epochs, the overall complexity for the training process can be expressed as $\mathcal{O}(\eta a E \sum_{l=1}^L n_l n_{l-1})$. The aggregate computational complexity, accounting for both the generation of data and training, is represented by $\mathcal{O}(\eta(HM + aE \sum_{l=1}^L n_l n_{l-1}))$.

3.4.3 Mapping relation from demonstration to preference

The term π_w is the DWMORL agent's policy aligned to the preference weight w . Assuming w is determined by a user

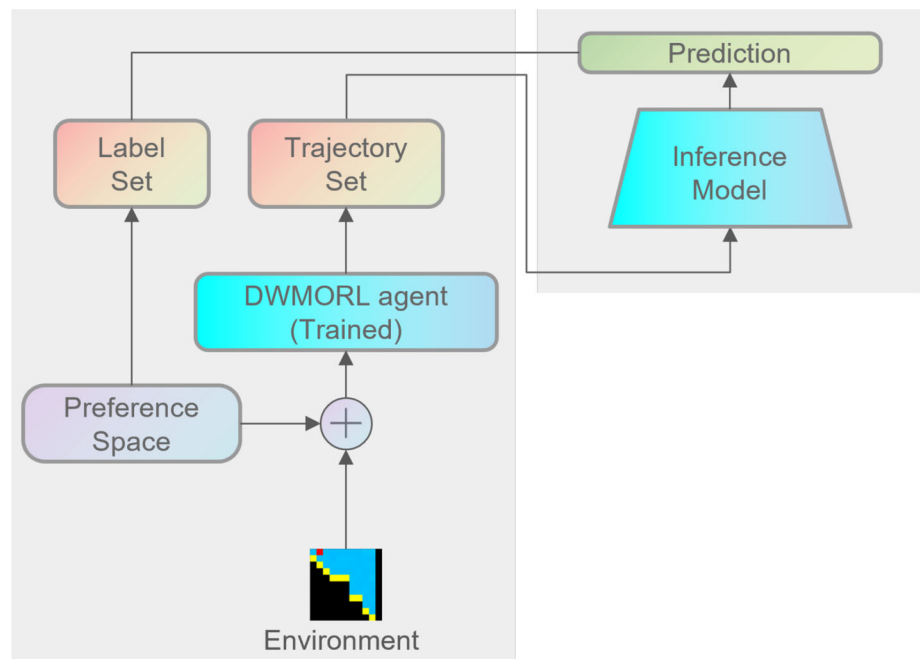
independently of the environment Env , the demonstration sampled from policy π_w is:

$$\tau_w \sim \pi_w \quad (2)$$

τ_w is the demonstration conditioned on the preference weight vector w . Equation 2 suggests a mapping relation between the demonstration and preference weight vector in environment Env .

An RL agent maps the policy to the demonstration [1, 8, 42], i.e., $\pi \rightarrow \tau$. This mapping relation is extended in MORL settings as the preference weight is mapped to the demonstration, i.e., $\pi_w \rightarrow \tau_w$. The mapping relation ($\pi \rightarrow \tau$) is reversible and introduced as an imitation learning (IL) [43, 44] where the policy is imitated from the demonstration, i.e., $\tau \rightarrow \pi$. In the MORL setting, this reversible mapping relation is expanded to the relationship between demonstrations and the preference weight, i.e., w . Our objective is to establish this mapping relation, denoted as $\tau_w \rightarrow w$. The mapping $\pi_w \rightarrow \tau_w$ involves only executing the policy under w for several episodes. When using a demonstration τ_w to infer w , the resultant singular, precise weight vector may be different from the label; however, it is still correct if it drops in the correct range of the preference space. Though the policy between the ground truth preference and w may be slightly different, they can still be deemed as equivalent under the specific circumstance. We term this as policy equivalence. We first give the formal definition of policy equivalence in the MORL setting then we relax the optimal demonstration assumption and give

Fig. 1 Train the DWPI model



the formal definition of policy δ -equivalence. We begin with the concept of corner weights proposed in [45].

The definition of policy equivalence has two primary purposes: (1) It supports the statement that an inference is considered correct if it yields a policy that is equivalent to the expected outcome, even if the inferred preference does not precisely match the ground truth. This approach recognizes the validity of inferences that lead to functionally comparable outcomes. (2) In scenarios where inference is based on a sub-optimal demonstration, a direct comparison between the inferred demonstration and the original sub-optimal one could lead to inaccuracies, resulting in a false negative. In such instances, if the demonstrations are found to be policy δ -equivalent, meaning they achieve a level of performance or outcome within a specified δ margin, the inference process is deemed to have been successful. This criterion allows for a more nuanced assessment of inference accuracy, accommodating the inherent variability and imperfection in demonstration-based inference.

Definition 1 Corner Weight [10, 45]

Let $\mathcal{V} = \{\mathbf{v}^{\pi_i}\}_i^n$ be a set of multi-objective value functions for n policies. Corner weights, $\mathbf{w}_c$, are weight vectors located at the vertices of a polyhedron P

$$P = \left\{ \mathbf{x} \in \mathbb{R}^{d+1} \mid \mathbf{V}^+ \mathbf{x} \leq 0, \sum_i w_i = 1, w_i \geq 0, \forall i \right\} \quad (3)$$

where $\mathbf{V}^+$ is the augmented matrix obtained by appending a column vector of -1 's to the matrix $\mathbf{V}$. Each vector $\mathbf{x} = (w_1, w_2, \dots, w_d, v_w)$ in P contains a weight vector $\mathbf{w}$ and its corresponding value v_w .

The corner weights represent the inner boundaries of the preference space and partition the preference weight space $\mathcal{W}$ into disjoint subsets $\{\mathcal{W}_i\}^{|\mathbf{w}_c|}$. When traversing those corner weights the optimal policy $\max_{\pi \in \Pi} v_w^\pi$ changes. If a policy is optimal for any one of the weight vectors in the corresponding subset, it is also optimal for all other weights within that subset.

In the context of MORL, the agent needs to balance between multiple objectives. When using linear preference weight inference, the cumulative reward vector provides a direct determination of the equivalence of two policies. In Definition 2, we give the general concept of policy equivalence.

Definition 2 Policy Equivalence

$\{\tau_i\}^{\pi_1}$ and $\{\tau_i\}^{\pi_2}$ are demonstration sets from two policies π_1 and π_2 . The metric d is used to measure the similarity¹ of demonstrations. If the two demonstration sets result in the same performance measured by metric d :

$$d(\{\tau_i\}^{\pi_1}, \{\tau_i\}^{\pi_2}) = 0 \quad (4)$$

π_1 and π_2 are equivalent under metric d , noted as:

$$\pi_1 \equiv_d \pi_2 \quad (5)$$

With Definitions 1 and 2, we introduce the definition of *CCS regional policy set* to refer to the policy set that is optimal for $\{\mathcal{W}_i\}^{|\mathbf{w}_c|}$.

Definition 3 CCS Regional Policy Set

For a corner weight $\mathbf{w}_c$, all equivalent policies that maximize the scalarized value under $\mathbf{w}_c$ constructs the CCS regional policy set $\Pi_{\mathbf{w}_c} \subseteq \Pi_{CCS}$.

According to Definition 1, $\mathbf{w}_c$ determines a region within which various weight vectors are considered equivalent in terms of their ability to generate a policy. Based on Definition 3, for any weight vector $\mathbf{w}_j$ in a subset bordered by $\mathbf{w}_c$, the policy $\pi_{\mathbf{w}_j}$ are equivalent to the policy $\pi_{\mathbf{w}_c}$ (Definition 2, taking cumulative reward feedback as the equivalence metric d). Therefore, the inference $\mathbf{w}_j$ is deemed correct as long as the ground truth weight also resides within this range.

3.4.4 Optimal demonstration relaxation

Note that the DWMORL policy π in Algorithm 4 is not necessarily optimal. The inference can still be accurate even if sub-optimal demonstrations are sampled by π . We now relax the optimal demonstration assumption. Using Definition 2, we define the concept of policy δ -equivalence.

Definition 4 Policy δ -Equivalence

Let $\{\tau_i\}^{\pi_1}$ and $\{\tau_i\}^{\pi_2}$ are demonstration sets from two policies π_1 and π_2 , and a metric d is used to measure the similarity of demonstrations. If the two demonstrations have a difference less than or equal to a certain threshold δ under metric d :

$$d(\{\tau_i\}^{\pi_1}, \{\tau_i\}^{\pi_2}) \leq \delta \quad (6)$$

π_1 and π_2 are considered equivalent under metric d .

$$\pi_1 \equiv_{d_\delta} \pi_2 \quad (7)$$

The predetermined value of δ should be sufficiently small to ensure that policies associated with distinct preference regions, denoted by $\mathbf{w}_c$, are clearly distinguishable.

¹ For example, one can employ the MSE between the cumulative rewards of demonstrations, or other metrics capable of quantifying the differences between two demonstrations, as a means of assessment.

Specifically, δ must be constrained to prevent the performance degradation of a sub-optimal policy from being considered to be a policy from another preference region. By using this δ threshold, when two preference weights—corresponding to a sub-optimal and an optimal policy, respectively—result in δ -equivalence, the preferences are considered to be effectively identical. This criterion aims to maintain a delicate balance: It must be tight enough to preserve the integrity of distinct preference delineations, yet flexible enough to accommodate minor discrepancies that do not materially affect the overall policy efficacy.

We have established a bidirectional mapping between optimal demonstrations and preference weights and elucidated the methodology for validating the correctness of inferences made by the DWPI model when provided with either optimal or sub-optimal demonstrations. This framework not only facilitates the understanding of how preferences can be accurately deduced from demonstrations but also outlines the criteria for assessing the fidelity of these inferences under different demonstration quality levels. This comprehensive approach ensures that the DWPI model's inferences are rigorously evaluated, enhancing its applicability and effectiveness in diverse scenarios where demonstration quality may vary.

4 Experiment setting

In this section, we provide our experimental settings, the baseline algorithms we used, the metrics of evaluation, and the sensitivity analysis setting. The preference weight vectors in all three simulations are normalized during training; the original weights are presented only to keep consistency with the literature (Fig. 2).

4.1 Experiment environment

4.1.1 Convex Deep Sea Treasure

CDST [24] is a variant of the Deep Sea Treasure (DST) environment [46], where the globally convex Pareto front is known. The state is the current agent position, while the action space consists of four movements: up, down, left, or right. Each episode begins with the agent positioned in the left top corner and ends when the agent reaches any of the yellow treasure grids. There are 10 different treasures, each corresponding to a distinct policy. The agent receives a reward vector comprising two elements: The first element is the time penalty of -1 per time step until the episode terminates, and the second element is the treasure reward. Balancing between the time penalty and treasure reward requires a two-dimensional preference vector. We

implement the DWMORL agent using DWMOTQ, i.e., Algorithm 1.

4.1.2 Traffic

The Traffic environment [8] has a collectible item on each of the upper corners. The yellow area is the road part where the red cars move. The car moves vertically in a random direction, and it reverses the direction when hit by the wall or the edge of the frame. The agent, starting from the left bottom corner, is not supposed to step on the road. If the agent steps on the road, it risks being hit by a car.

It must make a trade-off between obeying the traffic rules, maintaining traffic safety, and the time it takes to collect items. Each item is worth reward 1, stepping on the road causes reward -1 , being hit by a car causes reward -1 , hitting on the wall or the edge of the frame causes reward -1 , and the time penalty is -1 . In the work of Kallstrom et al. [8], four different simulation scenarios are given, leading to four typical behavior patterns. We retain the original preference setting from the literature to keep consistency. The elements of the preference vector are ordered as [*steps*, *item collection*, *break traffic rules*, *collisions*, *wall hitting*]. The DWMORL agent is implemented with Algorithm 2. The scenarios are shown below.

Always safe: [1, 50, 10, 50, 1]

The agent tries its best to avoid illegal behavior and collisions. It takes the longest but safest path to collect the two items.

Always fast: [10, 50, 10, 10, 1]

The agent tries to collect the two items as fast as possible. It thinks a detour is costly and therefore does not care about the risk of collision and breaking traffic rules by taking a short path.

Fast and safe: [5, 50, 0, 50, 1]

The agent cares about time consumption but also the safety of the path. It is encouraged to break traffic rules and walk on the yellow path to spend less time.

Slow and safe: [1, 50, 0, 50, 1]

The agent will walk on the road if there is a low risk of being hit by cars. It would rather take a longer path if the traffic condition is not ideal.

4.1.3 Item gathering

The Item Gathering environment [8] features three categories of collectibles in different colors. There is a hard-coded agent with fixed preferences that particularly favors the red item. At the start of each episode, the positions of the items are randomly initialized. The hard-coded agent starts at the top right corner and picks random routes to the red items.

The DWMORL agent exhibits an altruistic preference toward the hard-coded agent's collection of red items, which determines its cooperative or competitive behavior. The DWMORL agent obtains a reward of 1 that is linked to the objective of collecting an item of a particular color. An additional reward of 1 for its altruistic objective if the hard-coded agent collects a red item. The altruistic reward will be reversed based on the DWMORL agent's cooperation or competition behavior that depended on the symbol of the altruistic preference. The DWMORL agent receives a time penalty of -1 at each time step and a wall penalty of -1 upon collision.

Kallstrom et al. [8] examined four simulation scenarios to illustrate distinct behavior patterns of the DWMORL agent. We retain their setting to keep consistency. The preference vector's elements were arranged in the following order: [time penalty, wall penalty, collection of green items, collection of red items, collection of yellow items, and collection by the hard-coded agent].

Competitive: [1, 5, 10, 20, 10, -20]

The DWMORL agent's competitive behavior poses a challenge for the hard-coded agent in collecting red items.

Cooperative: [1, 5, 10, 20, 10, 20]

While both agents prioritize the collection of red items, the DWMORL agent's behavior is not characterized by vicious competition.

Fair: [1, 5, 20, 15, 20, 20]

The DWMORL agent prioritizes the collection of yellow and green items and is content with the success of the hard-coded agent.

Generous: [1, 5, 20, 0, 20, 20]

The DWMORL agent actively ignores and even avoids red items to promote the success of the hard-coded agent.

4.2 Baseline

We re-implement the PM method [21], and the MWAL method [22], as the two baseline algorithms to compare our DWPI algorithm against. The PM method randomly starts from a preference, trains a RL agent from scratch, and compares the resulting cumulative reward with the cumulative reward from the given demonstration. Similar to the PM method, MWAL method infers preferences by comparing the agent's and the given cumulative reward (from demonstration). Instead of randomly searching for preferences as PM, they use Eq. 8 to update the preference element, where w_n is the n th element of the preference, k is the number of feature elements, N is the total number of iterations, and μ_n and μ_{E_n} are the n th element of the feature expectation from current RL agent and the expert.

$$w_n \leftarrow w_n \cdot \left(1 + \sqrt{\frac{2 \log k}{N}} \right)^{-(\mu_n - \mu_{E_n})} \quad (8)$$

To make fair comparison, we used the same RL algorithm for PM and MWAL as the DWPI method.

4.3 Metrics

The research of PI is still at its early stage, yet a widely accepted metric is established. We therefore propose the following metrics to determine the accuracy of PI models.

4.3.1 Time efficiency

Efficient PI is crucial for practical applications that require rapid decision-making, especially in high-dimensional preference spaces. We evaluate the time consumption of the DWPI method and baselines by calculating the average time required for training phase and inference phase.

4.3.2 Inference accuracy

Known Pareto Front In CDST, the Pareto front is already known. In this scenario, the inference accuracy can be assessed by traversing through $\mathcal{W}^{\eta}$ and verifying whether the inferred preference weight falls within the correct range. The inference accuracy can then be calculated using the following formula, where $N_{correct}$ is the number of correct inferences:

$$ACC = \frac{N_{correct}}{|\mathcal{W}^{\eta}|} \quad (9)$$

Unknown Pareto Front The direct comparison is useful for evaluating inference accuracy when Pareto front is known. However, in real-life scenarios, a known Pareto front is a rare occurrence and a more frequent case is to do PI task with a unknown Pareto front.

Moreover, in MORL, the policy space often exhibits imbalanced clusters. A slight change in preference may lead to significant policy variations. This hinders the use of value-based comparison methods, e.g., MSE, for accuracy evaluation. In some cases, the ground truth and the inference may be very close to each other, but drop on two different sides of the boundary of two different policies (the corner weight). This indicates a plausible but incorrect inference and impact the evaluation adversely.

To address these two issues, comparing the cumulative rewards generated from the inferred preference weight with those generated from the true preference weight is a more reasonable evaluation metric of the inference accuracy. This approach is particularly effective in cases where a small preference change can result in a significant policy

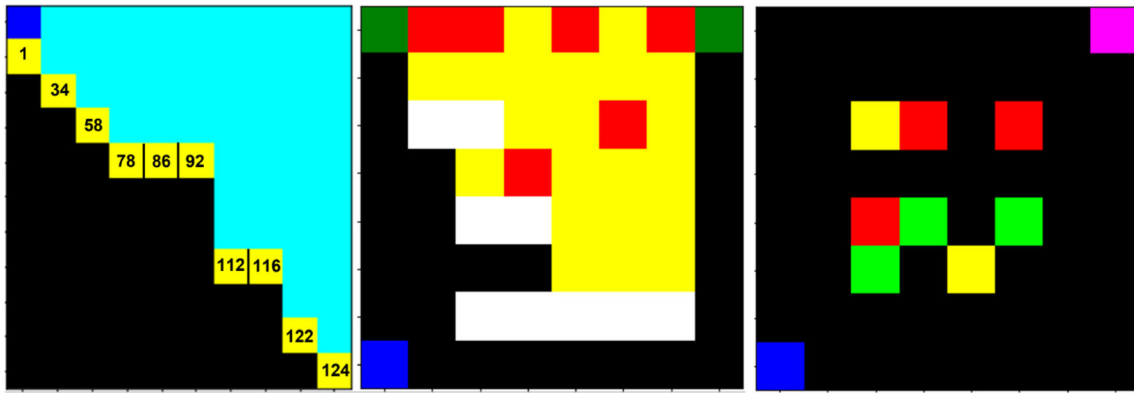


Fig. 2 CDST Environment (left): Agent in blue, treasures in yellow with numbers, walkable grids in light blue, unwalkable grids in black. Traffic Environment (middle): Agent in blue, item to collect in green, cars in red, roads in yellow, and walls in white. Item Gathering

Environment (right): Agent in blue, fixed-preference agent in pink, three categories collectable items in green, red and yellow. A fixed number of each category of items is randomly placed in the environment at the start of each episode

variation. If MSE is used to compute the difference between the cumulative rewards, we can use the following formula, where w is the ground truth and $\hat{w}$ is the inference:

$$ACC = \left(\sum_{\tau_w} r - \sum_{\tau_{\hat{w}}} r \right)^2 \quad (10)$$

4.4 Sensitivity analysis experiment setting

To evaluate the DWPI model when using different types of demonstration, we conducted a sensitivity analysis by assessing performance with different demonstration representations.

4.4.1 Varying demonstration representation

The DWPI model is trained with cumulative rewards (demonstration reward) by default. However, there are other representations of a demonstration, e.g., state demonstration. It is worthwhile to examine the difference in performance while different representations are used in PI.

In this section, we give the experiment settings to evaluate the performance of the DWPI model trained with state demonstrations. As the three simulation environments are all based on a grid world framework where the agent position is finite and discrete. We number the agent position and use a visiting frequency to denote the state demonstration. The state-frequency demonstration are calculated as:

$$\tau_s = \sum_{t=1}^{|\tau_s|} e_s(t) \cdot I_{|S|}^T \quad (11)$$

where $I_{|S|}$ is an identity matrix with shape $|S| \times |S|$, $e(s_t)$ is the one-hot vector that has one in the position

corresponding to state s_t . The state demonstrations are averaged from 100 demonstrations, similar to the reward demonstrations. The training hyperparameters are the same as the reward demonstration setting, except for the input layer's structure.

5 Results and analysis

5.1 Time efficiency

To ensure a fair comparison, we terminate the baselines once the error of PI converges. The time consumption of DWPI includes not only the PI process but also the training process of both the DWMORL model and the DWPI model.

The results are shown in Fig. 3. The figure illustrates that the DWPI algorithm demonstrated superior time efficiency compared to the baseline PM and MWAL algorithms.

Specifically, in the CDST simulation, DWPI was 94.83% faster than PM and 88.35% faster than MWAL; in the Traffic simulation, DWPI was 88.41% faster than PM and 68.02% faster than MWAL; and in the Item Gathering simulation, DWPI was 81.67% faster than PM and 49.85% faster than MWAL.

The results show that the DWPI algorithm is significantly more time-efficient for PI tasks in these three simulations compared to the baseline algorithms.

5.2 Inference accuracy

The overall inference accuracy comparison between the DWPI algorithm and baselines is shown in Fig. 4.

In CDST simulation, the direct comparison of inference and ground truth is adopted as the Pareto front is already

known. Compared with the PM and MWAL algorithms, the DWPI algorithm shows 20% and 42% higher inference accuracy, respectively. Additionally, our approach achieves perfect accuracy (100%) in inference tasks for demonstrations with a 50% stochastic ratio. Further information regarding the results of the CDST simulation can be found in Sect. 5.2.1.

Inference error is used for accuracy evaluation in the Traffic environment and the Item Gathering environment as their Pareto fronts are not known. The inference is operated on the resulted demonstrations of the four typical preference entries; the stochastic ratio is also 50% to evaluate the algorithms' robustness. In the Traffic simulation, DWPI achieves 94.91% lower inference error than the PM, and 98.12% lower inference error than the MWAL method. In the Item Gathering simulation, DWPI achieves 75.24% lower inference error than the PM and 75% lower inference error than the MWAL method.

It is worth noting that the MWAL algorithm consistently demonstrated the lowest inference accuracy across both CDST and Traffic simulations and similar performance as PM in Item Gathering simulation. This is attributed to the fact that the MWAL algorithm tends to perform poorly in situations where there are sub-optimal demonstrations and a stochastic environment. In contrast, the PM, while is not time-efficient, exhibits more robust performance than MWAL due to its use of a "sample-and-tryout" pattern. This enables the PM to make relatively good inferences even in situations with sub-optimal demonstrations. Our DWPI algorithm surpasses both MWAL and PM in terms of time efficiency and inference accuracy. In the upcoming sections, we will outline the specific details and outcomes of each simulation.

5.2.1 DST environment

In the CDST simulation, we compare the DWPI algorithm to baseline algorithms according on their inference accuracy. A more detailed comparison is presented in Fig. 5. Both PM and MWAL perform well when demonstrations are going to Treasures 124 and 122 because they are belong to the two largest policy regions. However, the performance of MWAL and PM deteriorates within policy regions mapped by the middle area of preference space. This is because those preference regions are more subtle and small, and even a small change in preference weight may incur a totally different policy. In the case of stochastic sub-optimal demonstration, while PM has a chance to infer a near-correct preference vector by sample and tryout, MWAL suffers from the difference between the sub-optimal demonstration and the optimal demonstration. When the preference range expands, both MWAL and PM exhibit improved inference accuracy, as demonstrated in

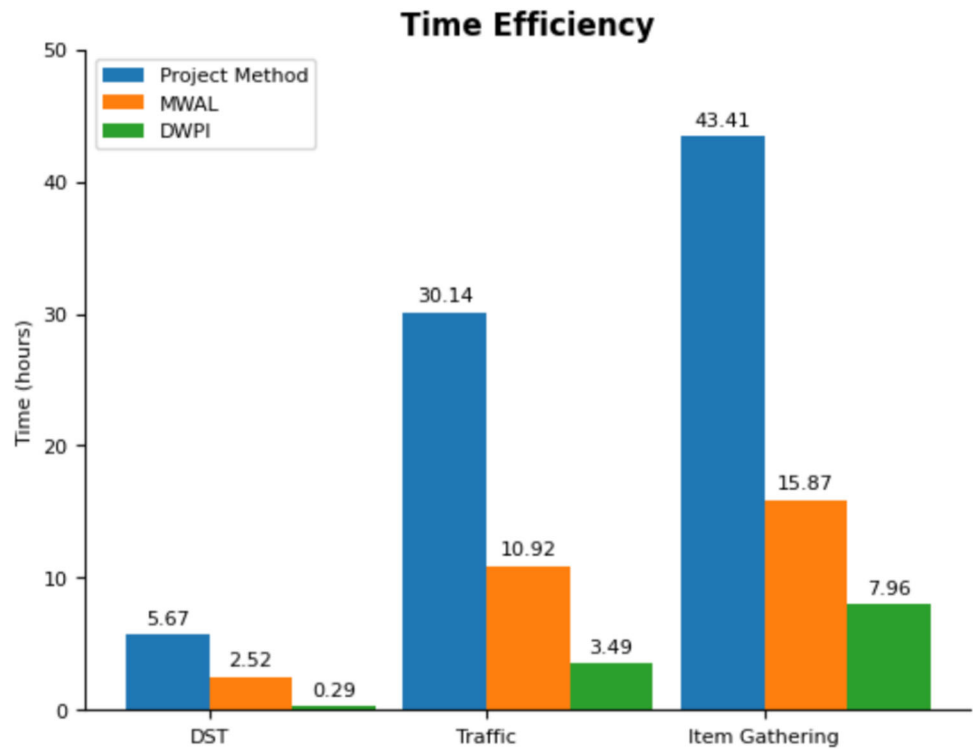
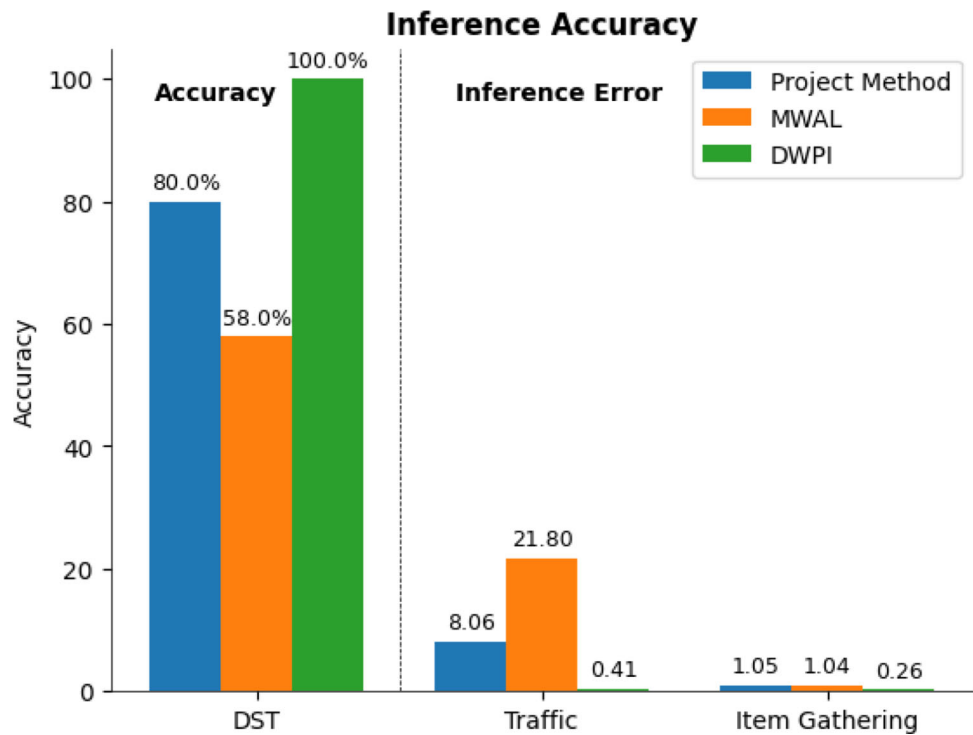
the Treasure-1 category, represented by the purple region. Over the two baselines, our DWPI outstands with 100% accuracy in CDST simulation which is shown in the right-most Figure.

5.2.2 Traffic environment

Figure 6 illustrates the outcome of PI in Traffic simulation. We assessed the inference accuracy in four common scenarios: always safe, always fast, fast safe, and slow safe. The red color represents the ground truth of the preference weight, while the green, blue, and purple colors represent the inference of DWPI, the PM, and MWAL, respectively. It is worth noting that the weights are in line with the experiment setting used by Kallstrom et al. [8], and normalization during training or inference would not affect their generality. The y-axis is the preference weight, and the x-axis labels are the objectives, i.e., [steps, item collection, break traffic rules, collisions, wall hitting]. The numeric inference results are put in the title of each subplot.

Based on the results depicted in Fig. 6, both the MWAL and PM approaches are able to capture the general shape of the true preference; however, there are some deviations to decrease the performance. In the "always fast" case, the MWAL method reduces too much the preference for avoiding collision and hitting the wall, while the PM shows two peaks in the preference distribution, despite the two preference weights actually being a downslope. The MWAL method performs well in the deterministic "always safe" case, where the agent picks the safest road and avoids breaking rules or being hit by a car in a grid world environment. However, the MWAL method is brittle in cases with stochasticity, such as "always fast," "fast safe," and "slow safe." On the other hand, the PM performs reasonably well in most cases and is not very sensitive to stochasticity, but it suffers from time efficiency and performs poorly in the "always fast" case.

The poor performance of baselines in the "always fast" case is due to the high stochasticity of the environment when the demonstrator wants to be always fast, and the agent goes on a road with many cars. This introduces significant stochastic noise to the reward demonstration, causing both baselines to almost fail to infer the preference weight. In contrast, the DWPI model, which can be trained by stochastic demonstrations, shows robustness in not only the "always fast" case but also perfectly infers the preference for the other three cases. As shown in Figs. 3, 4, and 6, our DWPI model can almost perfectly infer the preference weights within the shortest time.

Fig. 3 Time Efficiency Comparison**Fig. 4** Inference accuracy Comparison

5.2.3 Item gathering environment

Figure 7 showcases the results of the Item Gathering simulation where we evaluate the inference performance of the DWPI, PM, and MWAL for the competitive,

cooperative, fair, and generous behavior patterns of the agent. Similar to Fig. 6, the ground truth is shown in red while the results from the three methods are shown in green, blue, and purple. The preference weights are consistent with the ones used in literature [8], and

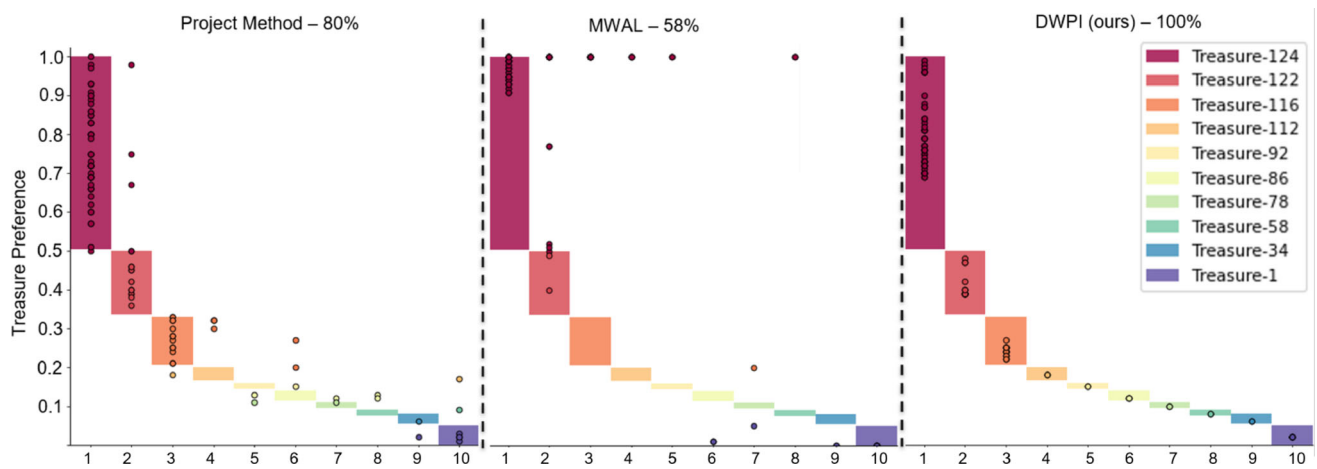


Fig. 5 Inference Result CDST—Stochastic demonstration Ratio—50%

normalization is applied for generality. The x-axis represents the objectives such as time penalty, wall penalty, and item collections. The numeric inferences are displayed in the subplot title.

The results in Fig. 7 indicate that our DWPI model can capture not only the general shape of the true preference but also demonstrate high accuracy numerically, including negative weights. In contrast, the PM can only capture the rough shape of the ground truth and fails to infer the negative weight for the competitive behavior pattern, and cannot provide distinguishable results for the cooperative and fair behavior patterns. Due to the high stochasticity of the environment where items are randomly dropped at the start of the episode, MWAL cannot give accurate inferences and even converges to incorrect preference weights.

5.3 Varying demonstration representation sensitivity analysis

In this section, we present the results of sensitivity tests, where we evaluate the inference performance of DWPI for different categories of demonstration. We run each evaluation for 5 times with different random seeds.² The results of inference from reward demonstration and state demonstration are shown in Fig. 8. Similar to Fig. 4, the metric of CDST simulation is the direct accuracy while the metric for Traffic simulation and Item Gathering simulation is inference error.

The inference accuracy from the reward demonstrations is better than the result from state demonstrations in CDST environment. However, in Traffic environment the state demonstrations are more efficient and effective for inference while the variance of state demonstration is slightly larger. In the Traffic simulation, when using reward

demonstration, there is an outlier which should be caused by the high stochasticity of the environment. In the Item Gathering environment, the two representations of demonstration achieve competitive result.

From the difference between the results from the two different types of demonstrations, we have two assumptions. (1) The results obtained from the simulation environments used in our study show that reward demonstration is a more effective descriptor than state demonstration to quantitatively infer the preference. However, as in Traffic and Item Gathering, the state demonstration is more effective to infer the preference that incur less variance of the resulted policy. (2) The inference performance of state demonstration improves when the environment becomes more stochastic. This may because that when the environment becomes more stochastic, even under the same preference, the agent may achieve very different rewards. However, the state sequence is more reliable to link with the preference in this scenario.

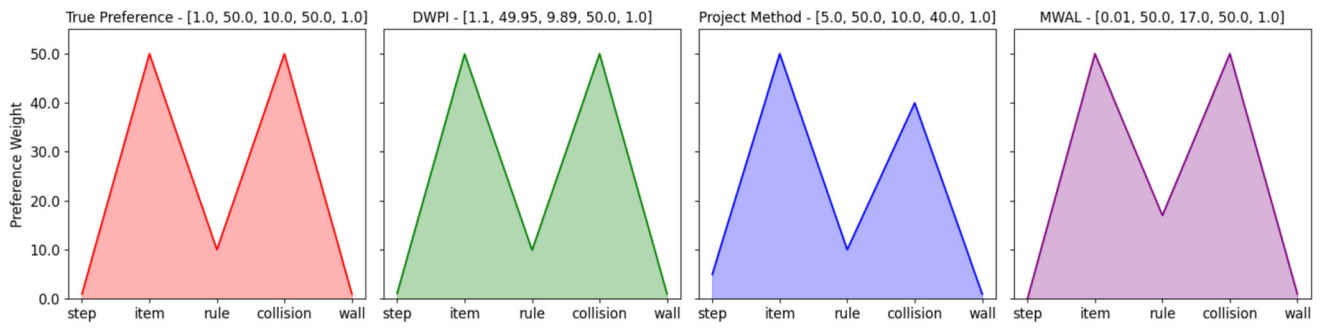
5.4 Discussion

We evaluated the performance of our proposed method against two baseline methods, PM and MWAL, in three different simulations: DST, Traffic, and Item Gathering. Our proposed DWPI approach demonstrated superior time efficiency and inference accuracy in all three simulations compared to the baseline methods.

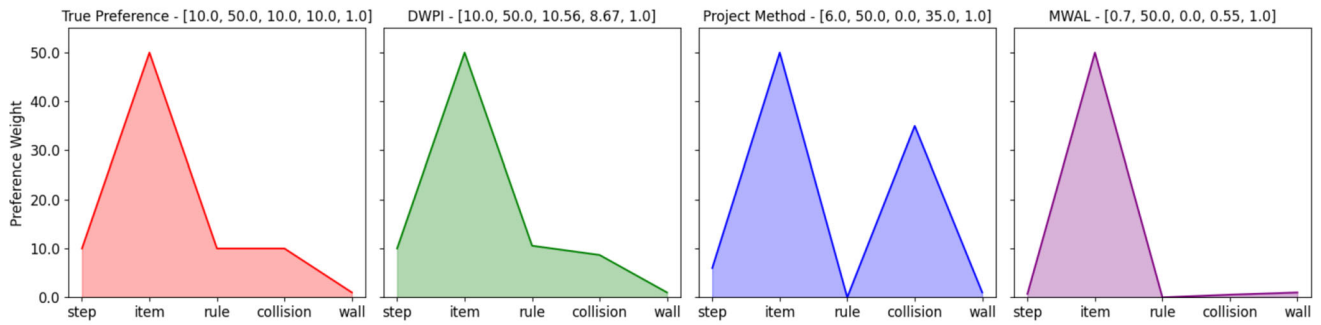
In the time efficiency comparison, our DWPI algorithm outperformed the PM and MWAL in all three simulations. In the inference accuracy comparison, our DWPI algorithm demonstrated higher accuracy in DST, Traffic, and Item Gathering simulations compared to the baseline methods.

We also conducted a sensitivity analysis to evaluate the performance of our DWPI algorithm for different categories of demonstration representation. The results indicate

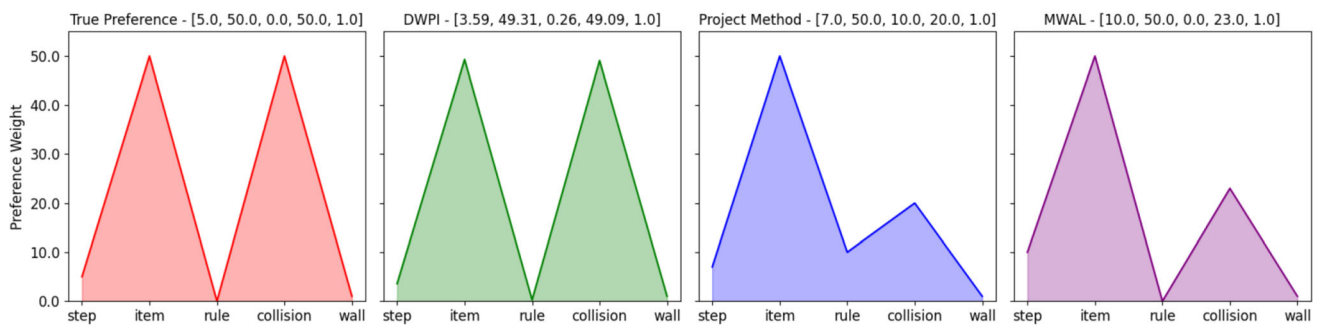
² Random seeds: 42, 17, 2, 24, 96.



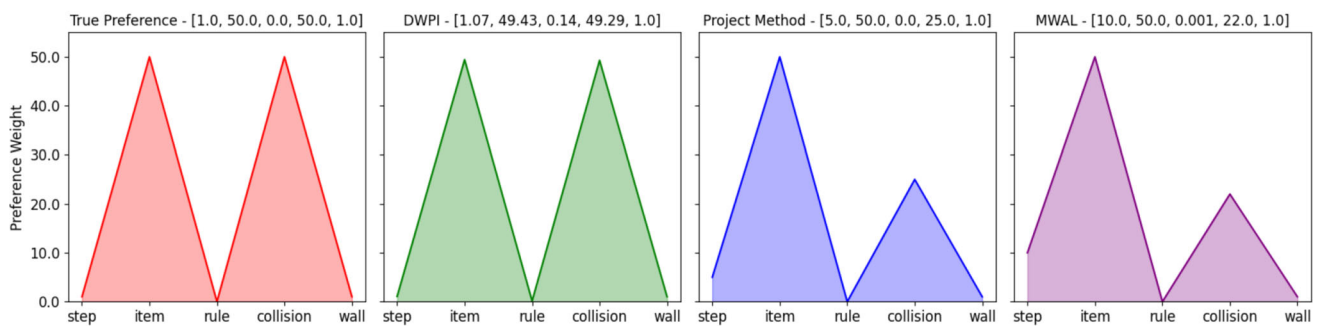
(a) Always Safe



(b) Always Fast

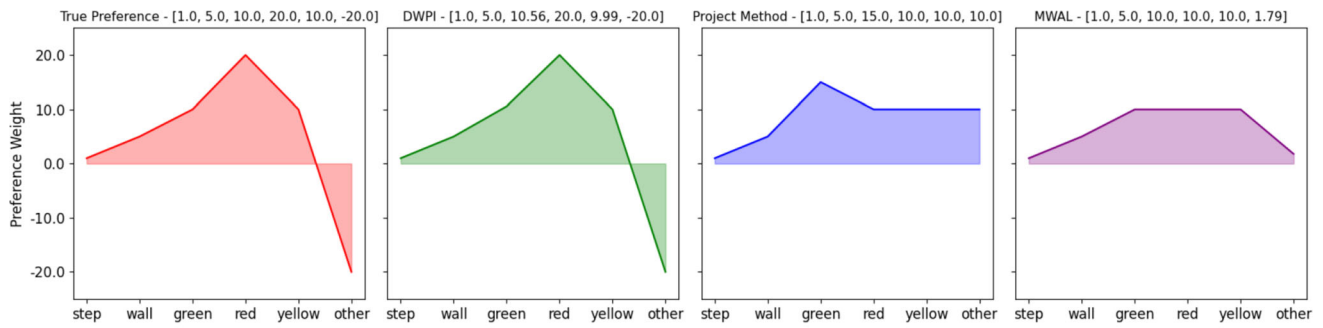


(c) Fast Safe

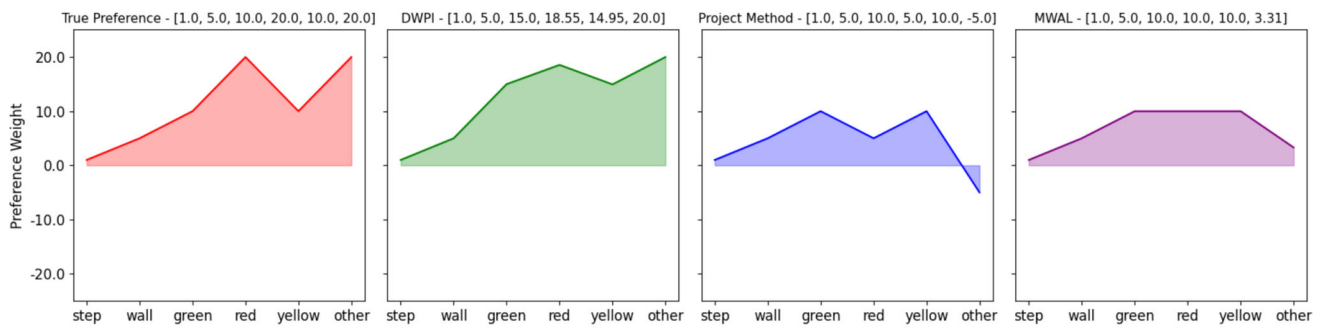


(d) Slow Safe

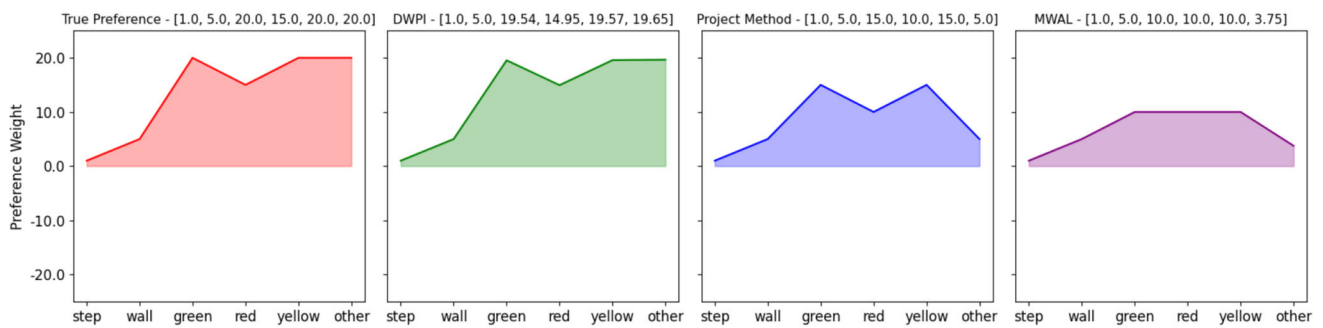
Fig. 6 Inference Result Traffic—Stochastic demonstration Ratio—50%



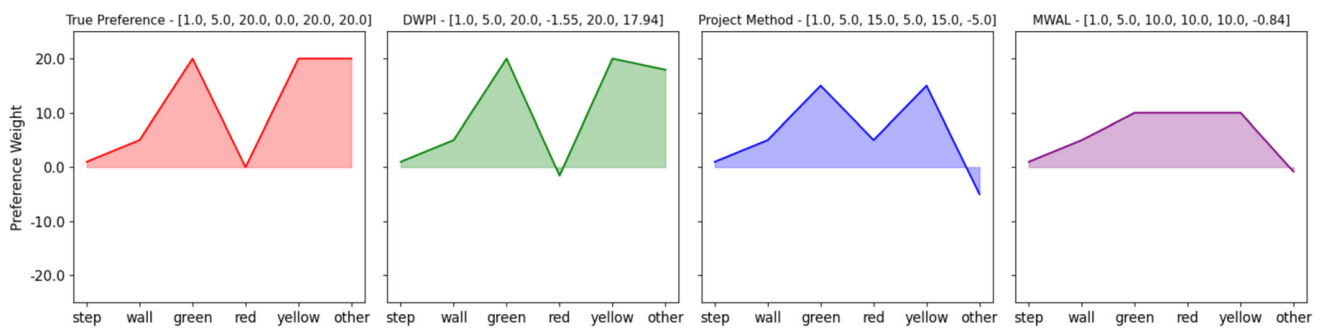
(a) Competitive



(b) Cooperative



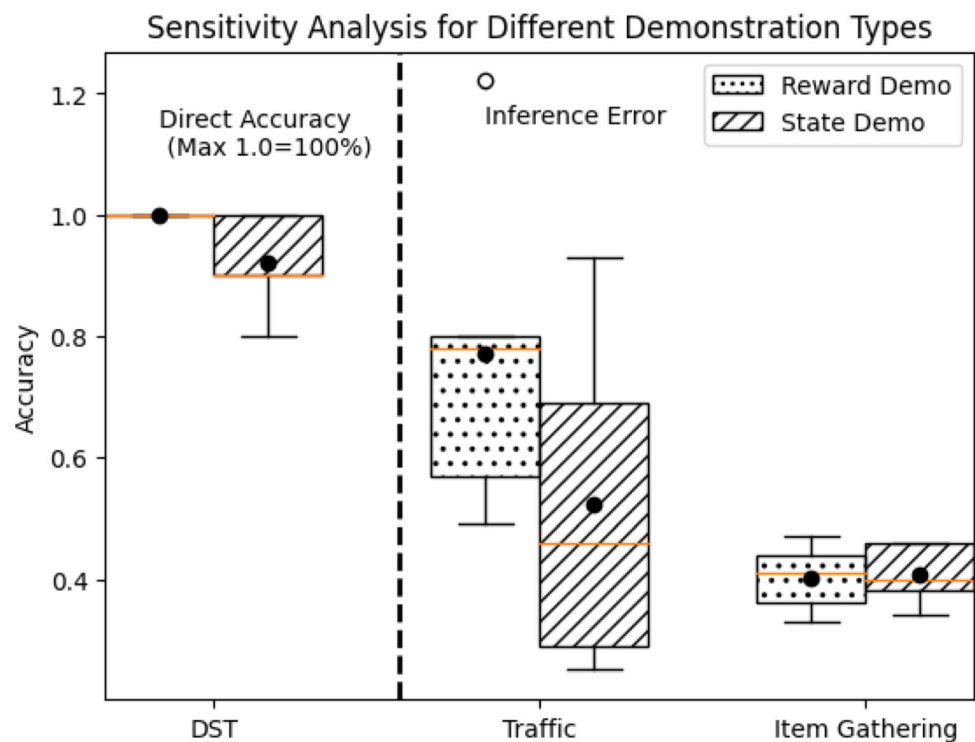
(c) Fair



(d) Generous

Fig. 7 Inference Result Item Gathering—Stochastic demonstration Ratio—50%

Fig. 8 Inference Accuracy Comparison Varying demonstration



that the reward demonstration is more effective in inferring preference than the state demonstration in all three simulations. The difference between the two representations becomes less pronounced as the dynamic nature of the environment becomes more stochastic, which is an important consideration when applying this method to real-world problems.

6 Conclusion

We introduce a novel approach “dynamic weight-based preference inference” for PI in multi-objective reinforcement learning. It is implemented based on a dynamic weight multi-objective reinforcement learning agent. The proposed algorithm outperforms existing baseline approaches in terms of accuracy and computational efficiency. It is capable of inferring user preferences even for sub-optimal demonstrations.

- The DWPI algorithm, a time-efficient, robust, query-free, and high-accuracy method for PI task in MORL settings. The trained model simply does a forward pass to infer the preference of a demonstration.
- Three distinct environments are used to evaluate the ability of generalization of the DWPI. Each of the three environments features a varying number of objectives.
- We provide a proof of correctness for the using a FNN for PI tasks and the complexity analysis of our

algorithm. We also introduce a metric to evaluate the accuracy of the inference process.

- To deliberately generate sub-optimal demonstrations, we use an energy-based model. This is done for augmenting the training dataset.
- We conduct a sensitivity analysis of DWPI model to evaluate the performance under different demonstration representations.

We aim to explore real human demonstrations in future work. Furthermore, incorporating representative learning techniques to improve the representation of the demonstration is another promising way to improve this work. Moreover, several avenues for future research are identified, including evaluating the DWPI algorithm in multi-agent environments, extending the algorithm to handle nonlinear utility functions, and more metrics to evaluate the accuracy of inference and the application in real-world problems.

Data availability We have released the code of this work on <https://github.com/JLu2022/DWPI>.

Declarations

Conflict of interest This research is funded by an Irish Research Council Government of Ireland Postgraduate Scholarship (GOIPG/2022/2140). All authors disclosed no relevant relationships.

References

- Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Belle-mare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533
- Lu J, Mannion P, Mason K (2023) Go-explore for residential energy management. In: *European Conference on Artificial Intelligence*, pp 133–139. Springer
- Kober J, Bagnell JA, Peters J (2013) Reinforcement learning in robotics: a survey. *The International Journal of Robotics Research* 32(11):1238–1274
- Hayes CF, Rădulescu R, Bargiacchi E, Källström J, Macfarlane M, Reymond M, Verstraeten T, Zintgraf LM, Dazeley R, Heintz F, Howley E, Irissappane AA, Mannion P, Nowé A, Ramos G, Restelli M, Vamplew P, Roijers DM (2022) A practical guide to multi-objective reinforcement learning and planning. *Autonom Agents Multi-Agent Syst* 36(1):26
- Mannor S, Shimkin N (2001) The steering approach for multi-criteria reinforcement learning. *Adv Neural Inform Process Syst* 14
- Tesauro G, Das R, Chan H, Kephart J, Levine D, Rawson F, Lefurgy C (2007) Managing power consumption and performance of computing systems using reinforcement learning. *Adv Neural Inform Process Syst* 20
- Van Moffaert K, Drugan MM, Nowé A (2013) Scalarized multi-objective reinforcement learning: Novel design techniques. In: *2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pp 191–199. IEEE
- Källström J, Heintz F (2019) Tunable dynamics in agent-based simulation using multi-objective reinforcement learning. In: *Adaptive and Learning Agents Workshop (ALA-19) at AAMAS, Montreal, Canada, May 13–14*, pp 1–7
- Yang R, Sun X, Narasimhan K (2019) A generalized algorithm for multi-objective reinforcement learning and policy adaptation. *Adv Neural Inform Process Syst* 32
- Alegre LN, Bazzan AL, Roijers DM, Nowé A, da Silva BC (2023) Sample-efficient multi-objective learning via generalized policy improvement prioritization. *arXiv preprint arXiv:2301.07784*
- Lu J, Mannion P, Mason K (2023) Inferring preferences from demonstrations in multi-objective reinforcement learning: a dynamic weight-based approach. *arXiv preprint arXiv:2304.14115*
- Lu J (2023) Preference inference from demonstration in multi-objective multi-agent decision making. In: *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pp 2961–2963
- Lu J, Mannion P, Mason K (2024) Inferring preferences from demonstrations in multi-objective residential energy management. *arXiv preprint arXiv:2401.07722*
- Hwang M, Weihs L, Park C, Lee K, Kembhavi A, Ehsani K (2023) Promptable behaviors: Personalizing multi-objective rewards from human preferences. *arXiv preprint arXiv:2312.09337*
- Benabbou N, Perny P (2015) Incremental weight elicitation for multiobjective state space search. In: *Twenty-Ninth AAAI Conference on Artificial Intelligence Vol. 29(1)*
- Zintgraf LM, Roijers DM, Linders S, Jonker CM, Nowé A (2018) Ordered preference elicitation strategies for supporting multi-objective decision making. *arXiv preprint arXiv:1802.07606*
- Benabbou N, Leroy C, Lust T (2020) An interactive regret-based genetic algorithm for solving multi-objective combinatorial optimization problems. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 2335–2342
- Shao H, Cohen L, Blum A, Mansour Y, Saha A, Walter MR (2023) Eliciting user preferences for personalized multi-objective decision making through comparative feedback. *arXiv preprint arXiv:2302.03805*
- Ng AY, Russell SJ, et al (2000) Algorithms for inverse reinforcement learning. In: *ICML*, vol 1, pp 2
- Ziebart BD, Maas AL, Bagnell JA, Dey AK, et al (2008) Maximum entropy inverse reinforcement learning. In: *AAAI*, vol 8, pp 1433–1438. Chicago, IL, USA
- Ikenaga A, Arai S (2018) Inverse reinforcement learning approach for elicitation of preferences in multi-objective sequential optimization. In: *2018 IEEE International Conference on Agents (ICA)*, pp 117–118. IEEE
- Takayama N, Arai S (2022) Multi-objective deep inverse reinforcement learning for weight estimation of objectives. *Artificial Life and Robotics*, pp 1–9
- Barrett L, Narayanan S (2008) Learning all optimal policies with multiple criteria. In: *Proceedings of the 25th International Conference on Machine Learning*, pp 41–47
- Mannion P, Devlin S, Mason K, Duggan J, Howley E (2017) Policy invariance under reward transformations for multi-objective reinforcement learning. *Neurocomputing* 263:60–73
- Van Moffaert K, Nowé A (2014) Multi-objective reinforcement learning using sets of pareto dominating policies. *J Mac Learn Res* 15(1):3483–3512
- Mossalam H, Assael YM, Roijers DM, Whiteson S (2016) Multi-objective deep reinforcement learning. *arXiv preprint arXiv:1610.02707*
- Abels A, Roijers D, Lenaerts T, Nowé A, Steckelmacher D (2019) Dynamic weights in multi-objective deep reinforcement learning. In: *International Conference on Machine Learning*, pp 11–20. PMLR
- Castelletti A, Pianosi F, Restelli M (2012) Tree-based fitted q-iteration for multi-objective markov decision problems. In: *The 2012 International Joint Conference on Neural Networks (IJCNN)*, pp 1–8. IEEE
- Roijers DM, Zintgraf LM, Nowé A (2017) Interactive thompson sampling for multi-objective multi-armed bandits. In: *International Conference on Algorithmic Decision Theory*, pp 18–34. Springer
- Castelletti A, Pianosi F, Restelli M (2013) A multiobjective reinforcement learning approach to water resources systems operation: Pareto frontier approximation in a single run. *Water Resour Res* 49(6):3476–3486
- Khamis MA, Gomaa W (2014) Adaptive multi-objective reinforcement learning with hybrid exploration for traffic signal control based on cooperative multi-agent framework. *Eng Appl Artif Intell* 29:134–151
- Ferreira PVR, Paffenroth R, Wyglinski AM, Hackett TM, Bilén SG, Reinhart RC, Mortensen DJ (2017) Multi-objective reinforcement learning-based deep neural networks for cognitive space communications. In: *2017 Cognitive Communications for Aerospace Applications Workshop (CCAA)*, pp 1–8. IEEE
- Lu J, Mannion P, Mason K (2022) A multi-objective multi-agent deep reinforcement learning approach to residential appliance scheduling. *IET Smart Grid* 5(4):260–280
- Hejna III DJ, Sadigh D (2023) Few-shot preference learning for human-in-the-loop rl. In: *Conference on Robot Learning*, pp 2014–2025. PMLR
- Haarnoja T, Tang H, Abbeel P, Levine S (2017) Reinforcement learning with deep energy-based policies. In: *International Conference on Machine Learning*, pp 1352–1361. PMLR
- Haarnoja T, Zhou A, Abbeel P, Levine S (2018) Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *International Conference on Machine Learning*, pp 1861–1870. PMLR

37. Chen L, Paleja R, Gombolay M (2021) Learning from suboptimal demonstration via self-supervised reward regression. In: Conference on Robot Learning, pp 1262–1277. PMLR
38. Brown DS, Goo W, Niekum S (2020) Better-than-demonstrator imitation learning via automatically-ranked demonstrations. In: Conference on Robot Learning, pp 330–359. PMLR
39. Beliaev M, Shih A, Ermon S, Sadigh D, Pedarsani R (2022) Imitation learning by estimating expertise of demonstrators. In: International Conference on Machine Learning, pp 1732–1748. PMLR
40. Cybenko G (1989) Approximation by superpositions of a sigmoidal function. *Math Control Sig Syst* 2(4):303–314
41. Hornik K (1991) Approximation capabilities of multilayer feed-forward networks. *Neural Netw* 4(2):251–257
42. Sutton RS, Barto AG (1999) Reinforcement learning: an introduction. Cambridge: MIT press
43. Ho J, Ermon S (2016) Generative adversarial imitation learning. *Adv Neural Inform Process Syst* 29
44. Osa T, Pajarinen J, Neumann G, Bagnell JA, Abbeel P, Peters J, *et al* (2018) An algorithmic perspective on imitation learning. *Found Trends® Robot* 7(1-2):1–179
45. Roijers DM (2016) Multi-objective decision-theoretic planning. *AI Matters* 2(4):11–12
46. Vamplew P, Dazeley R, Berry A, Issabekov R, Dekker E (2011) Empirical evaluation methods for multiobjective reinforcement learning algorithms. *Mach Learn* 84(1):51–80

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.